

Question 1: Exception Handling - Robust Calculator

```

class InvalidOperationError(Exception):
    pass

def calculator(num1, num2, operation):
    try:

        if not isinstance(num1, (int, float)):
            raise TypeError("First number must be numeric.")

        if not isinstance(num2, (int, float)):
            raise TypeError("Second number must be numeric.")

        # Arithmetic Operations
        if operation == "+":
            result = num1 + num2

        elif operation == "-":
            result = num1 - num2

        elif operation == "*":
            result = num1 * num2

        elif operation == "/":
            result = num1 / num2

        else:
            raise InvalidOperationError(
                f"Operation '{operation}' is not supported."
            )

        print("Result:", result)

    except ZeroDivisionError:
        print("Error: Cannot divide by zero!")

    except TypeError as e:
        print("Error:", e)

    except OverflowError:
        print("Error: Result is too large!")

    except InvalidOperationError as e:
        print("Error: InvalidOperationError -", e)

    finally:
        print("[Finally block executed - Operation logged]")

try:

    num1 = float(input("Enter first number: "))
    num2 = float(input("Enter second number: "))
    operation = input(
        "Enter operation (+, -, *, /): "
    )

    calculator(num1, num2, operation)

except ValueError:
    print(
        "Error: Invalid input! "
        "Please enter a valid number."
    )

```

```

Enter first number: 10
Enter second number: 0
Enter operation (+, -, *, /): /
Error: Cannot divide by zero!
[Finally block executed - Operation logged]

```

Question 2: File Operations - Text File Manager

```
import os
file_name = "sample.txt"

try:

    print("=== Text File Manager ===\n")

    print(f"1. Creating new file '{file_name}'...")

    lines = [
        "Line 1: Hello World\n",
        "Line 2: Python Programming\n",
        "Line 3: File Operations\n",
        "Line 4: Error Handling\n",
        "Line 5: End of file\n"
    ]

    with open(file_name, "w") as file:
        file.writelines(lines)

    print("File created successfully.")

    print("\n2. Writing content...")
    print(f"Content written: {len(lines)} lines")

    print("\n3. Reading entire file:\n")

    with open(file_name, "r") as file:
        content = file.read()

    print(content)

    print("Reading line by line:\n")

    with open(file_name, "r") as file:

        for line in file:
            print(line.strip())

    print("\nReading first 20 characters:\n")

    with open(file_name, "r") as file:
        chars = file.read(20)

    print(chars)

    print("\nUsing r+ mode:")

    with open(file_name, "r+") as file:

        current_content = file.read()

        print("Current file length:",
              len(current_content),
              "characters")

    file_size = os.path.getsize(file_name)

    with open(file_name, "r") as file:

        all_lines = file.readlines()

    line_count = len(all_lines)

    word_count = 0

    for line in all_lines:
        word_count += len(line.split())

    print("\n4. File Statistics:")

    print(f"Size: {file_size} bytes")
    print(f"Lines: {line_count}")
```

```

print(f"Words: {word_count}")

print("\n5. Appending new content...")

append_lines = [
    "\nLine 6: Appended Content",
    "\nLine 7: More Data"
]

with open(file_name, "a") as file:

    file.writelines(append_lines)

print("New content appended.")

print("\n6. Reading updated file:\n")

with open(file_name, "r") as file:

    updated_content = file.read()

print(updated_content)

except FileNotFoundError:

    print(
        "Error: File not found."
    )

except PermissionError:

    print(
        "Error: Permission denied."
    )

except IOError as e:

    print(
        f"I/O Error: {e}"
    )

except Exception as e:

    print(
        f"Unexpected Error: {e}"
    )

```

=== Text File Manager ===

1. Creating new file 'sample.txt'...
File created successfully.

2. Writing content...
Content written: 5 lines

3. Reading entire file:

Line 1: Hello World
Line 2: Python Programming
Line 3: File Operations
Line 4: Error Handling
Line 5: End of file

Reading line by line:

Line 1: Hello World
Line 2: Python Programming
Line 3: File Operations
Line 4: Error Handling
Line 5: End of file

Reading first 20 characters:

Line 1: Hello World

Using r+ mode:
Current file length: 114 characters

4. File Statistics:

```
Size: 114 bytes
```

```
Lines: 5
```

```
Words: 21
```

```
5. Appending new content...
```

```
New content appended.
```

```
6. Reading updated file:
```

```
Line 1: Hello World
```

```
Line 2: Python Programming
```

```
Line 3: File Operations
```

```
Line 4: Error Handling
```

```
Line 5: End of file
```

```
Line 6: Appended Content
```

```
Line 7: More Data
```

> Question 3: CSV File Processing with Error Handling

↳ 2 cells hidden

> Question 4: JSON Configuration Manager

↳ 1 cell hidden

> Question 5: Logging System with File Management

↳ 1 cell hidden

> Question 6: Data Backup System

↳ 1 cell hidden

> Question 7: File Validator and Processor

↳ 1 cell hidden

> Question 8: Comprehensive File Management System

↳ 1 cell hidden